

Neon Poker

Arquitectura de software propuesta

Compilación de la conversación de ChatGPT "Arquitectura app de póker". Documento unificado para usar como contexto constante de Codex.

Índice

- README
- 00-resumen-introduccion
- 01-arquitectura-general-recomendada
- 02-diagrama-textual-componentes
- 03-separacion-frontend-backend-motor
- 04-monolito-monorepo-microservicios
- 05-stack-recomendado
- 06-justificacion-tecnologias
- 07-servidor-vs-cliente
- 08-diseno-motor-poker
- 09-estado-tiempo-real
- 10-eventos-websocket
- 11-evitar-desincronizaciones
- 12-persistencia-hand-histories
- 13-reconstruir-mano-desde-eventos
- 14-diseno-lobby
- 15-diseno-sesiones-usuario
- 16-preparar-6max
- 17-preparar-modo-entrenamiento
- 18-preparar-solver-futuro
- 19-organizacion-carpetas-paquetes

- 20-tests-obligatorios
- 21-riesgos-tecnicos
- 22-decisiones-a-retrasar
- 23-no-construir-todavia
- 24-errores-tipicos
- 25-arquitectura-minima-viable-mvp
- 26-arquitectura-ideal-medio-plazo
- 27-prompt-para-codex
- 28-roadmap-fases
- 29-orden-implementacion
- 30-backlog-sprints
- 31-tests-por-fase
- 32-comprobaciones-codex-por-tarea
- 33-implementar-vs-preparar
- 34-riesgos-por-fase
- 35-checklist-revision-codex

Arquitectura Software — Neon Poker (contexto para Codex)

Esta carpeta contiene la arquitectura propuesta por ChatGPT para el proyecto **Neon Poker** (app educativa/play-money de Texas Hold'em online). Cada archivo es una sección autocontenida para que Codex tenga contexto constante sobre las decisiones técnicas.

Origen

Conversación de ChatGPT: "Arquitectura app de póker". Volcado tal cual fue propuesto, sin alteraciones.

Índice

Propuesta principal de arquitectura

- 00-resumen-introduccion.md — Resumen e introducción general
- 01-arquitectura-general-recomendada.md — Arquitectura general recomendada
- 02-diagrama-textual-componentes.md — Diagrama textual de componentes
- 03-separacion-frontend-backend-motor.md — Separación frontend / backend / motor / shared
- 04-monolito-monorepo-microservicios.md — Monolito modular vs monorepo vs microservicios
- 05-stack-recomendado.md — Stack recomendado definitivo
- 06-justificacion-tecnologias.md — Justificación de cada tecnología
- 07-servidor-vs-cliente.md — Qué vive en servidor y qué en cliente
- 08-diseno-motor-poker.md — Diseño del motor de póker
- 09-estado-tiempo-real.md — Manejo de estado en tiempo real (TableActor)
- 10-eventos-websocket.md — Diseño de eventos WebSocket
- 11-evitar-desincronizaciones.md — Cómo evitar desincronizaciones
- 12-persistencia-hand-histories.md — Persistencia de hand histories (event sourcing)
- 13-reconstruir-mano-desde-eventos.md — Reconstrucción determinista de manos
- 14-diseno-lobby.md — Diseño del lobby
- 15-diseno-sesiones-usuario.md — Diseño de sesiones de usuario
- 16-preparar-6max.md — Preparar la app para 6-max sin construirlo todavía
- 17-preparar-modo-entrenamiento.md — Preparar modo entrenamiento
- 18-preparar-solver-futuro.md — Preparar solver simplificado futuro
- 19-organizacion-carpetas-paquetes.md — Organización de carpetas y paquetes
- 20-tests-obligatorios.md — Tests obligatorios
- 21-riesgos-tecnicos.md — Riesgos técnicos principales
- 22-decisiones-a-retrasar.md — Decisiones que conviene retrasar

- 23-no-construir-todavia.md — Qué NO construir todavía
- 24-errores-tipicos.md — Errores típicos en una app de póker online
- 25-arquitectura-minima-viable-mvp.md — Arquitectura mínima viable para el MVP
- 26-arquitectura-ideal-medio-plazo.md — Arquitectura ideal a medio plazo
- 27-prompt-para-codex.md — Prompt exacto para pasarle a Codex

Roadmap de ejecución para Codex

- 28-roadmap-fases.md — Roadmap por fases (Fase 0 → Fase 4)
- 29-orden-implementacion.md — Orden exacto de implementación (30 prompts)
- 30-backlog-sprints.md — Backlog dividido por sprints
- 31-tests-por-fase.md — Tests que Codex debe crear por fase
- 32-comprobaciones-codex-por-tarea.md — Qué debe comprobar Codex al terminar cada tarea
- 33-implementar-vs-preparar.md — Implementar ahora vs dejar preparado vs no construir
- 34-riesgos-por-fase.md — Riesgos principales por fase
- 35-checklist-revision-codex.md — Checklist final para revisar el trabajo de Codex

Cómo usar este contexto con Codex

1. Apunta AGENTS.md del repositorio a esta carpeta o copia los archivos al repo.
2. En cada tarea, recuerda a Codex que respete los principios de los archivos 08, 11, 15, 21 y 35.
3. Usa 27-prompt-para-codex.md como prompt inicial del proyecto.
4. Sigue el orden de 29-orden-implementacion.md.

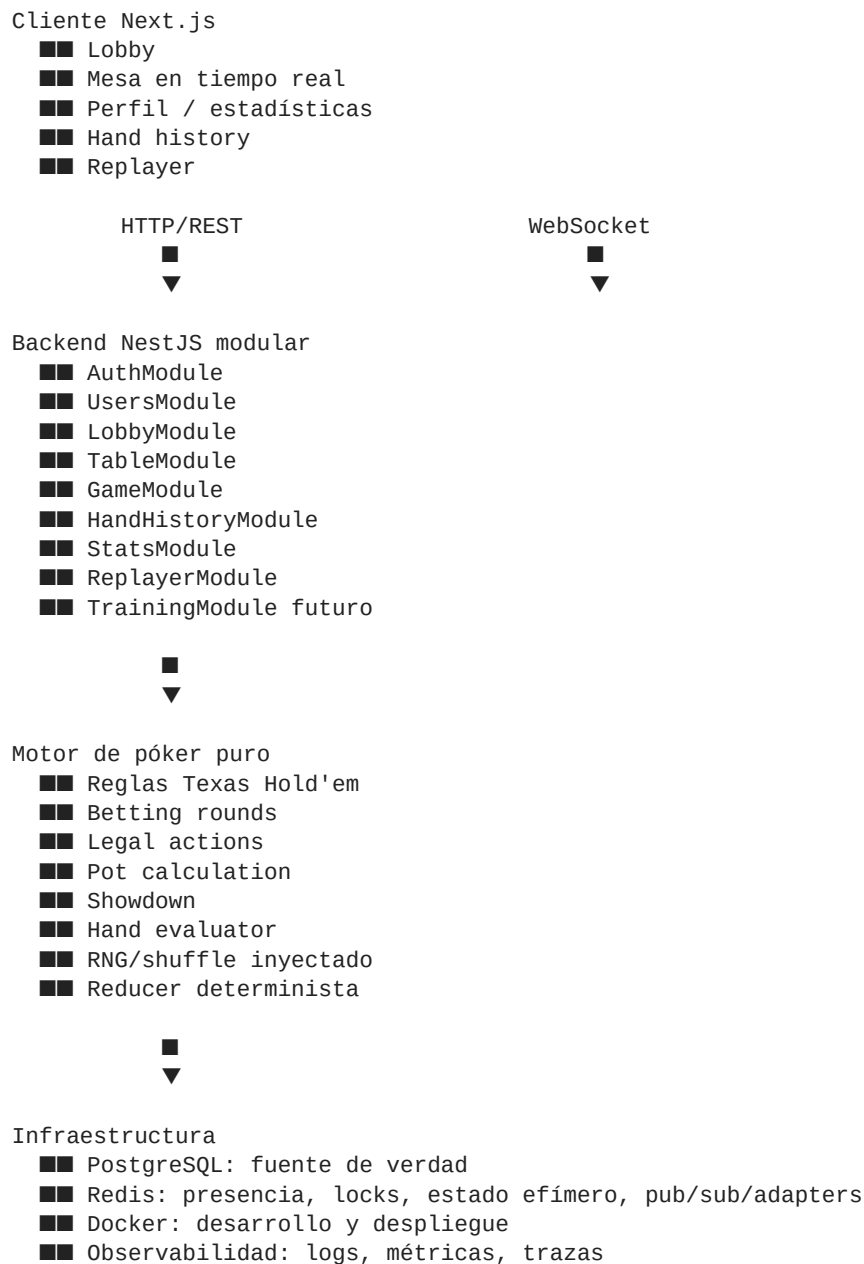
Resumen e introducción — Neon Poker

Tu stack tentativo es muy bueno para "Neon Poker". No lo cambiaría radicalmente. Lo convertiría en una arquitectura más explícita: monorepo, backend como monolito modular, motor de póker puro y separado, PostgreSQL como fuente de verdad, Redis como coordinación/cache temporal, WebSocket autoritativo desde el servidor y hand history basada en eventos.

La decisión más importante no es "Next vs otra cosa", sino esta: el cliente nunca decide el estado del juego. El cliente solo muestra, anima y envía intenciones. El servidor valida, ejecuta el motor, persiste eventos y emite snapshots/eventos filtrados por usuario.

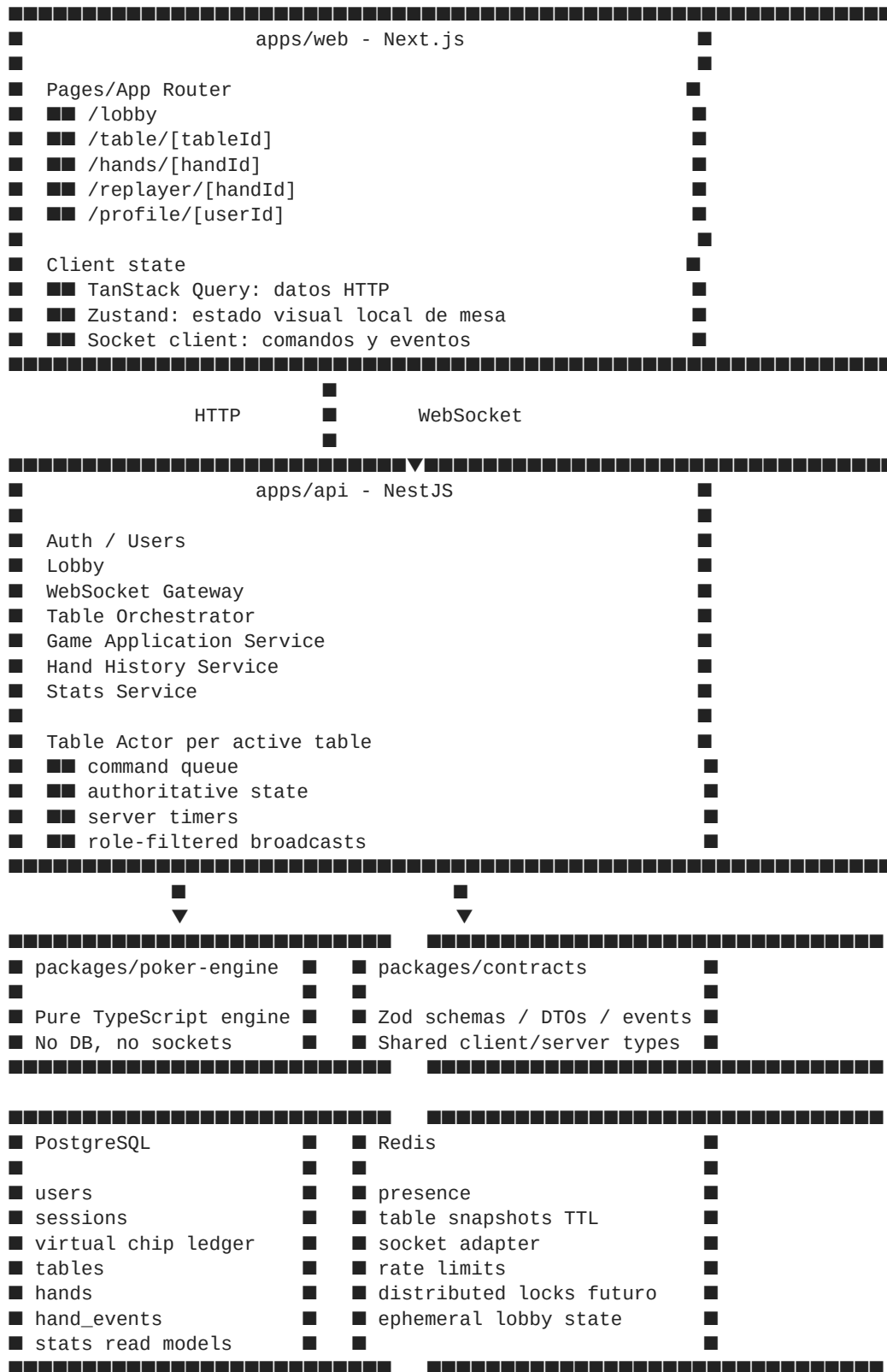
1. Arquitectura general recomendada

Arquitectura recomendada:



Next.js encaja bien para lobby, perfil, historial, replayer y rutas de producto; la mesa debe ser un componente cliente altamente interactivo. Next.js App Router trabaja con Server Components y Client Components, lo que permite separar vistas SSR/estáticas de zonas interactivas. NestJS también encaja bien porque sus gateways WebSocket integran conceptos de inyección de dependencias, guards, pipes e interceptores, y soporta tanto `socket.io` como `ws`.

2. Diagrama textual de componentes



3. Separación ideal entre frontend, backend, motor y módulos compartidos

El frontend debe contener presentación, animaciones, interacción, replayer visual y estado local no autoritativo. Puede saber cuáles son las acciones legales que el servidor le envía, pero no debe calcular ni decidir si una acción es válida.

El backend debe contener autenticación, sesiones, ownership de mesas, validación de comandos, ejecución del motor, timers, persistencia, hand histories, estadísticas, control de reconexión y filtrado de información privada.

El motor de póker debe vivir como paquete puro TypeScript, sin NestJS, sin PostgreSQL, sin Redis, sin WebSocket y sin `Date.now()` ni `Math.random()`. Debe recibir dependencias inyectadas: RNG, reloj, IDs y configuración de reglas.

Los módulos compartidos deben contener contratos, tipos, validadores y utilidades puras. Para validar payloads de cliente/servidor, Zod es una buena opción porque permite definir esquemas TypeScript-first y validar datos no confiables.

Separación sugerida:

```
apps/  
  web/  
  api/  
  
packages/  
  poker-engine/  
  contracts/  
  db/  
  ui/  
  config/  
  test-utils/
```

4. Monolito modular, monorepo, microservicios o híbrida

La decisión recomendada es: monorepo + monolito modular + motor separado como paquete puro.

No empezaría con microservicios. En una app de póker online, los problemas difíciles son consistencia, turnos, timers, reconexión, privacidad de cartas y hand history. Los microservicios añaden latencia, coordinación distribuida y más formas de desincronizarse.

Monorepo no significa monolito. El monorepo sirve para compartir tipos, contratos, motor, tests y configuración. El monolito modular sirve para mantener una sola frontera transaccional y reducir complejidad operativa.

Arquitectura híbrida a medio plazo: mantener el monorepo, pero separar procesos cuando tenga sentido:

apps/api	HTTP + auth + lobby + gateway
apps/table-runner	actores de mesa
apps/worker	stats, análisis, jobs, replayer exports
apps/web	frontend

Pero no separaría bases de datos ni haría microservicios independientes al principio.

5. Stack recomendado definitivo

Stack recomendado:

Frontend:

- Next.js App Router
- React
- TypeScript
- Tailwind CSS
- Framer Motion
- TanStack Query
- Zustand
- Socket.IO client

Backend:

- NestJS
- TypeScript
- Socket.IO Gateway inicialmente
- Zod para contratos/validación
- Drizzle ORM + PostgreSQL
- Redis con ioredis
- OpenTelemetry para trazas/métricas futuras

Motor:

- TypeScript puro
- Vitest
- fast-check para property-based testing
- Fixtures de manos
- RNG inyectado
- Hand evaluator testado exhaustivamente

Infra:

- Docker Compose en desarrollo
- Docker en producción
- PostgreSQL
- Redis
- GitHub Actions o CI equivalente

Tests:

- Vitest
- Playwright
- Testcontainers o entorno Docker de integración

Socket.IO es preferible para el MVP porque aporta rooms, reconexión, acknowledgements y adapters. Sus rooms son una abstracción del servidor para emitir a subconjuntos de sockets, útil para `table:{id}` y `user:{id}`. Si más adelante necesitas menor overhead y control total, puedes migrar partes a ws, que NestJS también soporta mediante adapter.

PostgreSQL debe ser la fuente de verdad porque necesitas transacciones, consistencia y consultas sobre usuarios, mesas, manos, eventos, sesiones, estadísticas y ledger de fichas ficticias. PostgreSQL documenta niveles de aislamiento transaccional y soporta JSON/JSONB, lo que encaja bien con eventos versionados en hand history.

Redis debe usarse para presencia, locks, snapshots temporales, rate limiting y fanout, no como fuente final de hand history. Redis Pub/Sub tiene semántica at-most-once, así que si un suscriptor falla o se desconecta, el mensaje puede perderse definitivamente. Redis Streams sí puede servir como log append-only para jobs o distribución interna, pero la fuente de verdad de manos debe seguir siendo

PostgreSQL.

Drizzle ORM es mi elección para este caso porque mantiene el código cerca de SQL, funciona con PostgreSQL y encaja bien con esquemas explícitos, eventos append-only y transacciones. Prisma también sería una alternativa válida si priorizas DX, generación de cliente y onboarding rápido; su documentación lo presenta como ORM open-source con acceso type-safe a PostgreSQL y otras bases de datos.

6. Justificación de cada tecnología

Next.js: buena elección para producto web completo, no solo para la mesa. Lobby, perfil, hand history y replay se benefician de routing, layouts y separación entre servidor/cliente.

React: adecuado para UI de mesa, componentes reutilizables, animaciones, estado visual y replay.

TypeScript: obligatorio en este producto. Reduce errores en contratos, eventos, estados de motor y payloads WebSocket.

Tailwind: bueno para construir rápido una UI consistente y con apariencia profesional.

Framer Motion: útil para animaciones de cartas, fichas y transiciones. No debe controlar lógica de juego; solo animar estados ya confirmados por servidor.

NestJS: buena elección para backend modular, DI, guards, pipes, gateways y separación por módulos.

Socket.IO: recomendable para MVP por rooms, reconexión, acknowledgements y adapters. Para escalar Socket.IO a varios servidores, el Redis adapter reenvía paquetes entre servidores, aunque se siguen necesitando sticky sessions.

PostgreSQL: fuente de verdad transaccional.

Redis: estado efímero, presencia, rate limits, snapshots temporales y coordinación.

Drizzle: ORM explícito y cercano a SQL.

Zod: contratos runtime compartidos entre cliente y servidor.

Vitest: unit tests rápidos para motor y backend. Vitest se presenta como framework de testing moderno impulsado por Vite y también usable en backend.

Playwright: E2E para flujos reales de navegador; incluye runner, assertions, aislamiento, paralelización y tooling para apps modernas.

OpenTelemetry: recomendable a medio plazo para observabilidad porque permite generar y recolectar trazas, métricas y logs en JavaScript/Node.js.

7. Qué vive en servidor y qué puede vivir en cliente

Debe vivir en servidor:

- Barajado
- RNG
- Deck
- Cartas privadas
- Validación de acciones
- Legal actions reales
- Turn order
- Timers
- Blinds
- Pot calculation
- All-ins
- Showdown
- Awarding de botes
- Stack efectivo
- Ledger de fichas ficticias
- Hand history interna
- Estadísticas persistidas
- Control de sesiones
- Prevención de acciones duplicadas

Puede vivir en cliente:

- Render de mesa
- Animaciones
- Sonidos
- Preferencias visuales
- Estado de modales
- Vista local del replayer
- Historial público recibido del servidor
- Cálculos visuales no autoritativos
- Cuenta atrás basada en deadline del servidor

El cliente puede importar utilidades puras del motor para el replayer o para mostrar ayudas educativas, pero nunca debe recibir seed, deck completo, cartas privadas de otros jugadores ni estado interno del motor activo.

8. Diseño del motor de póker

El motor debe diseñarse como functional core:

```
type GameCommand =
  | { type: 'StartHand'; tableId: string }
  | { type: 'PlayerAction'; playerId: string; action: PokerAction }
  | { type: 'Timeout'; playerId: string }
  | { type: 'SeatPlayer'; playerId: string; seatIndex: number }
  | { type: 'StandUp'; playerId: string };

type DomainEvent =
  | { type: 'HandStarted'; handId: string; buttonSeat: number }
  | { type: 'BlindPosted'; playerId: string; amount: number }
  | { type: 'PrivateCardsDealt'; playerId: string; cards: Card[] }
  | { type: 'BoardCardsDealt'; street: Street; cards: Card[] }
  | { type: 'ActionRequested'; playerId: string; legalActions: LegalAction[]; deadlineAt: string }
  | { type: 'PlayerActed'; playerId: string; action: PokerAction }
  | { type: 'PotUpdated'; pots: Pot[] }
  | { type: 'ShowdownStarted' }
  | { type: 'HandEnded'; winners: Winner[] };

function decide(
  state: GameState,
  command: GameCommand,
  deps: EngineDeps
): DomainEvent[];

function applyEvent(
  state: GameState,
  event: DomainEvent
): GameState;
```

Reglas clave:

- decide() valida y produce eventos.
- applyEvent() aplica eventos de forma determinista.
- El motor no persiste.
- El motor no emite WebSockets.
- El motor no conoce usuarios reales, solo playerId/seatId.
- El motor no usa Math.random().
- El motor no usa Date.now().
- El motor no modifica estado externo.
- Cada cambio de estado debe ser reproducible.

Para el RNG: usa seed generado en servidor con fuente criptográfica, barajado Fisher-Yates y versión de algoritmo. Para reconstrucción exacta, persiste eventos de cartas repartidas o deck interno cifrado/privado. Para hand history pública, expón solo board, acciones y cartas reveladas/showdown.

El estado debe tener vistas filtradas:

```
toPublicTableView(state): PublicTableView
toPlayerTableView(state, playerId): PlayerTableView
toSpectatorView(state): SpectatorTableView
toInternalDebugView(state): InternalTableState
```

Esto evita filtrar cartas privadas accidentalmente.

9. Cómo manejar estado en tiempo real

Usa el patrón single-writer por mesa. Una mesa activa debe tener un único actor autoritativo:

```
TableActor(tableId)
  ■■ estado interno actual
  ■■ cola secuencial de comandos
  ■■ deadline/timers del turno
  ■■ seq incremental
  ■■ persistencia de eventos
  ■■ broadcast filtrado
```

Flujo de una acción:

1. Cliente envía comando: `game.action`.
2. Gateway autentica socket.
3. Gateway envía comando al `TableActor`.
4. `TableActor` verifica `tableId`, `handId`, `playerId`, `turno` y `expectedSeq`.
5. Motor produce `DomainEvents`.
6. Backend persiste eventos en PostgreSQL dentro de transacción.
7. Actor aplica eventos e incrementa `seq`.
8. Actor emite eventos/snapshot filtrado a cada usuario.
9. Cliente confirma, anima y actualiza vista.

Para el MVP, un único proceso NestJS puede alojar los actores. A medio plazo, los actores deberían vivir en `apps/table-runner`, y los gateways solo enrutarían comandos.

10. Diseño de eventos WebSocket

Separa comandos, eventos de dominio y eventos de transporte.

Comandos cliente → servidor:

```
lobby.subscribe
lobby.unsubscribe
table.join
table.leave
table.sit
table.stand
game.action
game.sitOutNextHand
game.requestSnapshot
handHistory.request
```

Eventos servidor → cliente:

```
lobby.snapshot
lobby.patch
table.snapshot
table.event
table.patch
game.actionRequired
game.actionAccepted
game.actionRejected
game.resyncRequired
connection.authExpired
connection.kicked
```

Envelope recomendado:

```
type WsEnvelope<T> = {
  type: string;
  schemaVersion: number;
  requestId?: string;
  idempotencyKey?: string;
  userId?: string;
  tableId?: string;
  handId?: string;
  seq?: number;
  serverTime: string;
  payload: T;
};
```

Comando de acción:

```
type GameActionCommand = {
  tableId: string;
  handId: string;
  expectedSeq: number;
  idempotencyKey: string;
  action:
    | { type: 'fold' }
    | { type: 'check' }
    | { type: 'call' }
    | { type: 'bet'; amount: number }
```

```
    | { type: 'raise'; amount: number }  
    | { type: 'allIn' };  
};
```

Evento de acción requerida:

```
type ActionRequiredEvent = {  
  tableId: string;  
  handId: string;  
  seq: number;  
  playerId: string;  
  legalActions: LegalAction[];  
  minRaise?: number;  
  maxRaise?: number;  
  callAmount?: number;  
  deadlineAt: string;  
};
```

No envíes "estado interno completo" sin filtrar. A cada socket se le debe mandar una vista específica para ese usuario.

11. Cómo evitar desincronizaciones

Reglas obligatorias:

- Servidor autoritativo siempre.
- El cliente nunca muta estado real.
- Cada evento de mesa lleva seq.
- El cliente solo aplica seq + 1.
- Si hay gap de secuencia, pide snapshot.
- Cada comando incluye expectedSeq.
- Cada comando incluye idempotencyKey.
- Cada snapshot incluye stateHash.
- Cada hand_event persistido incluye stateHashAfter.
- Las reconexiones reciben snapshot completo.
- Los timers se calculan en servidor.
- El cliente muestra countdown derivado de deadlineAt.

El cliente puede hacer UI optimista suave, por ejemplo deshabilitar botones tras pulsar, pero no debe asumir que la acción fue aceptada hasta recibir `game.actionAccepted` o el siguiente `table.event`.

Si un usuario se reconecta:

1. Autenticar socket.
2. Unir a room user:{userId}.
3. Detectar mesas activas del usuario.
4. Unir a room table:{tableId}.
5. Enviar table.snapshot personalizado.
6. Reanudar countdown desde deadlineAt.

12. Cómo persistir hand histories

Usaría event sourcing para las manos, no solo un resumen final.

Tablas principales:

```
users
sessions
virtual_chip_accounts
virtual_chip_ledger
```

```
tables
table_seats
table_sessions
```

```
hands
hand_events
hand_participants
hand_summaries
player_stats
```

hands:

```
id
table_id
game_type
max_seats
small_blind
big_blind
button_seat
started_at
ended_at
status
rules_version
engine_version
shuffle_version
final_state_hash
summary_jsonb
```

hand_events:

```
id
hand_id
seq
event_type
actor_player_id nullable
payload_jsonb
visibility
schema_version
created_at
state_hash_after
```

hand_participants:

```
hand_id
user_id
seat_index
starting_stack
```

```
ending_stack  
net_result  
hole_cards_encrypted nullable  
hole_cards_revealed boolean
```

Para eventos, jsonb es práctico porque los payloads evolucionan por tipo y versión, y PostgreSQL soporta almacenamiento JSON/JSONB con mecanismos de consulta.

Importante: no confundas hand history pública con hand history interna. Internamente puedes guardar eventos completos. Públicamente debes filtrar cartas privadas no reveladas.

13. Cómo reconstruir una mano desde eventos

Reconstrucción exacta:

1. Cargar hand.
2. Cargar hand_events ordenados por seq.
3. Crear initialState con rules_version, table config y participantes.
4. Aplicar applyEvent() secuencialmente.
5. Verificar stateHashAfter por evento.
6. Verificar final_state_hash al final.
7. Generar vista pública o vista personalizada para el usuario.

Pseudocódigo:

```
export function rebuildHand(events: DomainEvent[], initial: GameState): GameState {
  return events.reduce((state, event) => {
    const next = applyEvent(state, event);
    assertStateHash(next, event.stateHashAfter);
    return next;
  }, initial);
}
```

Para el replayer visual, no necesitas reconstruir todo el estado interno del deck. Necesitas una secuencia pública de acciones, board, cambios de stack, botes y cartas reveladas. Para auditoría interna, sí debes poder reconstruir todo.

14. Cómo diseñar el lobby

El lobby debe combinar HTTP inicial + updates WebSocket.

Flujo:

1. Cliente carga /lobby por HTTP.
2. Backend devuelve lista de mesas y estado resumido.
3. Cliente abre WebSocket y emite lobby.subscribe.
4. Servidor envía lobby.snapshot.
5. Cambios posteriores llegan como lobby.patch.

Modelo de mesa:

```
type LobbyTableView = {  
  tableId: string;  
  name: string;  
  game: 'NLHE';  
  format: 'HEADS_UP';  
  maxSeats: 2;  
  seatedCount: number;  
  waitingCount: number;  
  smallBlind: number;  
  bigBlind: number;  
  avgPot?: number;  
  handsPerHour?: number;  
  status: 'open' | 'playing' | 'paused';  
};
```

La configuración de mesas debe persistir en PostgreSQL. La presencia, ocupación live, waiting list y métricas efímeras pueden cachearse en Redis. Pero si Redis cae, el sistema debe poder reconstruir desde PostgreSQL y actores activos.

15. Cómo diseñar sesiones de usuario

Recomendación:

- Access token corto.
- Refresh token en cookie `httpOnly`, `secure`, `sameSite`.
- Tabla sessions en PostgreSQL.
- Redis opcional para revocación rápida/presencia.
- WebSocket autenticado en handshake.
- Socket middleware para validar usuario.
- Rooms: `user:{userId}`, `table:{tableId}`.

Socket.IO documenta middleware para autenticación/autorización y uso de JWT en conexión, lo que encaja con validar el usuario al abrir el socket.

Evitaría guardar tokens sensibles en `localStorage`. Para una app educativa no necesitas KYC, pagos ni nada relacionado con dinero real, pero sí necesitas seguridad básica porque hay cuentas, perfiles y estado persistente.

16. Cómo preparar la app para 6-max sin implementarlo todavía

No hardcodees heads-up.

Diseña desde el principio:

```
type TableConfig = {
  gameType: 'NLHE';
  maxSeats: 2 | 6;
  smallBlind: number;
  bigBlind: number;
  minBuyInBb: number;
  maxBuyInBb: number;
};

type Seat = {
  index: number;
  playerId?: string;
  stack: number;
  status: 'empty' | 'seated' | 'sittingOut';
};
```

Reglas:

- seats debe ser array genérico.
- buttonSeat debe ser índice, no player1/player2.
- action order debe calcularse por función.
- blinds deben resolverse por número de jugadores activos.
- UI debe renderizar asientos por maxSeats.
- pot model debe permitir varios ganadores.
- side pot module debe existir aunque el MVP heads-up no lo use intensivamente.

En heads-up hay reglas especiales de botón y blinds. Trátalas como una rama dentro de una función genérica, no como toda la arquitectura.

17. Cómo preparar la app para modo entrenamiento

El modo entrenamiento debe reutilizar el mismo motor.

Diseño:

```
TrainingModule
  ■■ ScenarioService
  ■■ TrainingSessionService
  ■■ BotDecisionService futuro
  ■■ FeedbackService futuro
  ■■ MistakeTaggingService futuro
```

Tipos:

```
type TrainingScenario = {
  id: string;
  title: string;
  tableConfig: TableConfig;
  initialStacks: SeatStack[];
  heroSeat: number;
  holeCards?: Card[];
  board?: Card[];
  actionHistory?: DomainEvent[];
  learningGoal: 'preflop' | 'cbet' | 'potOdds' | 'valueBet' | 'bluffCatch';
};
```

El entrenamiento no debe ser otra app. Debe ser otra fuente de comandos para el mismo motor. En vez de sockets reales con dos humanos, tienes hero + bot/coach + escenarios predefinidos.

18. Cómo preparar la app para solver simplificado futuro

No construyas el solver todavía. Prepara interfaces.

El motor debe exponer:

```
getLegalActions(state): LegalAction[]
applyActionForAnalysis(state, action): GameState
clonePublicState(state): PublicGameState
evaluateHand(cards, board): HandRank
calculateEquity(input): EquityResult futuro
```

Crea un paquete futuro:

```
packages/poker-analysis/
  ■■ equity/
  ■■ ranges/
  ■■ pot-odds/
  ■■ heuristics/
  ■■ simplified-solver/
```

El solver no debe bloquear la mesa en tiempo real. A medio plazo debe correr en worker separado. Para MVP educativo, empieza con análisis simple:

- pot odds
- equity Monte Carlo básica
- fuerza relativa de mano
- SPR
- tamaño de bote
- clasificación de acción: fold/call/raise
- explicación textual

No intentes resolver GTO real al principio.

19. Organización de carpetas y paquetes

Estructura recomendada:

```
neon-poker/  
  apps/  
    web/  
      app/  
      components/  
      features/  
        lobby/  
        table/  
        hand-history/  
        replayer/  
        profile/  
      lib/  
      styles/  
  
  api/  
    src/  
      main.ts  
      app.module.ts  
      modules/  
        auth/  
        users/  
        lobby/  
        tables/  
        game/  
        hand-history/  
        stats/  
        sessions/  
      realtime/  
        gateways/  
        socket-auth.middleware.ts  
        ws-event-bus.ts  
      infrastructure/  
        db/  
        redis/  
        config/  
        logging/  
      common/  
        guards/  
        pipes/  
        filters/  
  
  packages/  
    poker-engine/  
      src/  
        cards/  
        deck/  
        evaluator/  
        betting/  
        pots/  
        state/  
        commands/  
        events/  
        reducers/  
        rules/  
        rng/  
        views/  
      tests/  
        fixtures/
```

```
    property/

contracts/
  src/
    ws/
    http/
    events/
    schemas/
    dto/

db/
  src/
    schema/
    migrations/
    repositories/

ui/
  src/

config/
  src/

test-utils/
  src/

docker-compose.yml
package.json
pnpm-workspace.yaml
turbo.json
AGENTS.md
```

Módulos NestJS sugeridos:

```
AuthModule
UsersModule
SessionsModule
LobbyModule
TablesModule
GameModule
HandHistoryModule
StatsModule
RealtimeModule
```

20. Tests obligatorios

Tests del motor:

- Deck tiene 52 cartas únicas.
- Shuffle determinista con seed.
- No se reparten cartas duplicadas.
- Evaluador de manos correcto.
- Ranking de manos correcto.
- Blinds heads-up correctas.
- Orden de acción preflop heads-up correcto.
- Orden de acción postflop correcto.
- Fold termina mano correctamente.
- Check/check avanza street.
- Bet/call avanza street.
- Raise mínimo/máximo correcto.
- All-in correcto.
- Showdown correcto.
- Awarding correcto.
- Stack total conservado.
- Pot total conservado.

Tests de eventos:

- Cada comando válido produce eventos esperados.
- Cada comando inválido es rechazado.
- applyEvent es determinista.
- rebuildHand(events) reproduce finalState.
- stateHashAfter coincide.
- eventos privados no aparecen en vista pública.

Tests de backend:

- Auth HTTP.
- Auth WebSocket.
- Join table.
- Sit down.
- Start hand.
- Player action.
- Timeout server-side.
- Disconnect/reconnect.
- Doble click de acción con mismo idempotencyKey.
- Dos acciones simultáneas.
- Persistencia de hand_events.
- Ledger de fichas ficticias sin saldos negativos.

Tests de frontend:

- Lobby renderiza mesas.
- Mesa renderiza snapshot.
- Botones legales aparecen correctamente.
- Botones se deshabilitan si no es turno.
- Replayer reproduce eventos.

Tests E2E:

- Usuario A y Usuario B entran.
- Ambos se sientan.

- Se juega una mano completa.
- Se guarda hand history.
- Se abre replayer.
- Reconnect durante mano.
- Timeout de jugador.

Tests de seguridad:

- Un jugador no puede actuar fuera de turno.
- Un usuario no puede actuar por otro.
- Un espectador no recibe cartas privadas.
- Un cliente no puede mandar amount inválido.
- Un cliente no puede modificar stack.
- Un usuario no puede unirse a una mesa inexistente.

21. Riesgos técnicos principales

Los riesgos principales son:

- Bugs de betting rules.
- Desincronización cliente/servidor.
- Filtración de cartas privadas.
- Race conditions en acciones simultáneas.
- Timers inconsistentes.
- Reconexiones mal gestionadas.
- Hand histories incompletas.
- Redis usado como fuente de verdad.
- Microservicios prematuros.
- Estado de mesa duplicado en varias instancias.
- Stats calculadas desde estado parcial.
- UI optimista que contradice al servidor.
- Falta de fixtures de manos.
- No versionar eventos.
- No tener stateHash.

El riesgo más peligroso a medio plazo es correr varias instancias backend con actores de mesa en memoria sin resolver ownership. El Redis adapter de Socket.IO ayuda a emitir eventos entre servidores, pero no resuelve por sí solo quién es el único escritor de una mesa. Para eso necesitas un TableActor propietario, routing de comandos y, si escalas, fencing tokens o table-runner dedicado.

22. Decisiones que conviene retrasar

Retrasaría:

- Microservicios reales.
- Kubernetes.
- Multi-región.
- Solver complejo.
- 6-max completo.
- Torneos.
- Sit & Go.
- Rankings avanzados.
- Bots sofisticados.
- Chat global.
- Anti-collusion complejo.
- Marketplace de entrenamientos.
- App móvil nativa.
- GraphQL.
- Kafka.
- NATS.
- Diseño final de escalado horizontal.

No retrases:

- Motor testeable.
- Event sourcing de manos.
- Validación server-side.
- Contratos tipados.
- Reconexión.
- Privacidad de cartas.
- Idempotencia.
- Tests.

23. Qué NO construir todavía

No construiría todavía:

- Pagos.
- Depósitos.
- Retiradas.
- Rake.
- Premios convertibles.
- Cualquier economía conectada a dinero real.
- Solver GTO.
- 6-max real.
- Torneos.
- Sit & Go.
- Multi-table avanzado.
- Chat con moderación compleja.
- Sistema de amigos.
- Marketplace.
- Leaderboards con premios.
- Admin panel complejo.
- Motor anti-fraude avanzado.
- Multi-región.
- Serverless WebSocket.

Tampoco construiría un motor "rápido" sin tests. En póker, un pequeño bug de turn order, all-in o showdown puede invalidar todo el producto.

24. Errores típicos en una app de póker online

Errores comunes:

- Calcular reglas en el cliente.
- Usar `Math.random()` para barajar.
- No persistir eventos, solo resultado final.
- No distinguir evento de dominio y evento WebSocket.
- Enviar cartas privadas en snapshots públicos.
- No tener idempotencyKey.
- No manejar doble click.
- No manejar reconexión.
- Usar Redis como hand history.
- No versionar eventos.
- Hardcodear heads-up en todo el código.
- No probar all-ins.
- No probar folds preflop.
- No probar botón/blinds heads-up.
- No hacer fixtures de manos reales.
- No separar motor de NestJS.
- Mezclar animaciones con lógica.
- No tener actor único por mesa.
- No tener auditoría de fichas ficticias.

25. Arquitectura mínima viable para el MVP

MVP recomendado:

apps/web

- Login/register
- Lobby
- Mesa heads-up
- Hand history básica
- Replayer básico
- Perfil simple

apps/api

- Auth
- Users
- Sessions
- Lobby
- TableActor in-process
- Game service
- Hand history
- Stats mínimas

packages/poker-engine

- NLHE heads-up
- Barajado
- Blinds
- Preflop/flop/turn/river
- Fold/check/call/bet/raise/all-in
- Showdown
- Awarding
- Rebuild desde eventos

packages/contracts

- WS events
- HTTP DTOs
- Zod schemas

PostgreSQL

- users
- sessions
- tables
- hands
- hand_events
- hand_participants
- virtual_chip_ledger

Redis

- presence
- lobby cache
- rate limits
- table snapshot TTL opcional

Para el MVP puedes correr una sola instancia de NestJS. Eso evita el problema de ownership distribuido de mesas. Aun así, diseñá TableActor como si pudiera moverse a otro proceso después.

26. Arquitectura ideal a medio plazo

Medio plazo:

```
apps/web
apps/api
apps/table-runner
apps/worker
```

```
packages/poker-engine
packages/contracts
packages/db
packages/poker-analysis
packages/ui
packages/config
```

Flujo medio plazo:

Cliente

```
■■ HTTP → API
■■ WS → Gateway
```

Gateway

```
■■ autentica
■■ enruta comando
■■ emite eventos al socket
```

Table Runner

```
■■ owns table actors
■■ ejecuta motor
■■ persiste hand_events
■■ genera snapshots
■■ publica cambios
```

Worker

```
■■ stats
■■ análisis
■■ exports
■■ tareas futuras
```

En este punto puedes tener varias instancias gateway, Socket.IO Redis adapter, sticky sessions, table ownership en Redis/PostgreSQL y workers para análisis. Socket.IO mantiene adapters oficiales para escalar broadcasts a múltiples servidores, incluyendo Redis adapter y Redis Streams adapter.

27. Prompt exacto para pasarle después a Codex

Para una tarea de este tamaño, conviene pedir primero plan, contexto y cambios verificables. La documentación de Codex recomienda dar buen contexto, pedir planificación en tareas complejas y usar `AGENTS.md` para instrucciones reutilizables del repositorio.

Prompt exacto:

Actúa como arquitecto principal y senior full-stack engineer. Quiero crear el MVP de "Neon Poker", una

Objetivo:

Construir una base de producción para una app heads-up de póker online con lobby, mesa en tiempo real,

Stack obligatorio:

- Monorepo con pnpm workspaces y Turborepo.
- Frontend: Next.js App Router + React + TypeScript + Tailwind + Framer Motion.
- Backend: NestJS + TypeScript.
- Tiempo real: Socket.IO sobre NestJS Gateway.
- Base de datos: PostgreSQL.
- ORM: Drizzle.
- Redis: presencia, lobby cache, rate limiting y estado efímero.
- Validación/contratos: Zod.
- Tests: Vitest para unit/integration, Playwright para E2E.
- Infra local: Docker Compose.

Arquitectura obligatoria:

- Monolito modular en backend, no microservicios por ahora.
- Motor de póker separado en packages/poker-engine.
- El motor debe ser TypeScript puro, sin NestJS, sin DB, sin Redis, sin WebSocket, sin Date.now() y sin
- El motor debe funcionar con command -> domain events -> reducer/applyEvent.
- El servidor es siempre autoritativo.
- El cliente nunca calcula ni decide el estado real de la mano.
- PostgreSQL es la fuente de verdad.
- Redis nunca debe ser la fuente de verdad de hand history.
- Las manos deben persistirse como eventos append-only.
- Cada evento de mano debe tener handId, seq, eventType, payload, schemaVersion y stateHashAfter.
- Debe existir reconstrucción determinista de una mano desde eventos.
- Deben existir vistas filtradas: public view, player view, spectator view.
- Nunca se deben enviar cartas privadas de otros jugadores al cliente.

Crea esta estructura:

```
apps/  
  web/  
  api/  
  
packages/  
  poker-engine/  
  contracts/  
  db/  
  ui/  
  config/  
  test-utils/
```

Implementa primero un MVP funcional, no toda la visión futura.

MVP funcional:

1. Auth básica con usuario registrado.
2. Lobby con lista de mesas heads-up.
3. Crear/unirse a mesa.
4. Sentarse/levantarse.

5. Iniciar mano cuando haya 2 jugadores sentados.

6. Motor NLHE heads-up:

- deck de 52 cartas únicas
- shuffle determinista con RNG inyectado
- blinds heads-up
- preflop/flop/turn/river
- fold/check/call/bet/raise/all-in
- showdown
- award pot

7. WebSocket events:

- lobby.subscribe
- lobby.snapshot
- lobby.patch
- table.join
- table.snapshot
- game.action
- game.actionRequired
- game.actionAccepted
- game.actionRejected
- table.event
- game.resyncRequired

8. Persistencia:

- users
- sessions
- tables
- hands
- hand_events
- hand_participants
- virtual_chip_accounts
- virtual_chip_ledger

9. Hand history básica.

10. Replayer básico desde eventos públicos.

11. Perfil con estadísticas mínimas:

- manos jugadas
- manos ganadas
- VPIP básico si es fácil
- resultado neto en fichas ficticias

Reglas de seguridad:

- No implementar dinero real.
- No implementar pagos.
- No implementar depósitos.
- No implementar retiradas.
- No implementar rake.
- No implementar premios convertibles.
- No aceptar stacks, cartas, pots ni legalActions enviados por el cliente.
- Validar todos los payloads con Zod.
- Usar idempotencyKey en acciones.
- Usar seq/expectedSeq para evitar desincronización.
- En reconexión, enviar snapshot completo filtrado para el usuario.
- No guardar tokens sensibles en localStorage.

Tests obligatorios:

- Unit tests del motor:
 - deck único
 - shuffle determinista
 - blinds heads-up
 - turn order preflop
 - turn order postflop
 - fold termina mano
 - check/check avanza street
 - bet/call avanza street
 - showdown correcto
 - conservación de fichas
 - reconstrucción desde eventos

- Tests de privacidad:
 - public view no contiene hole cards privadas
 - player view solo contiene cartas propias
- Integration tests backend:
 - join table
 - sit down
 - start hand
 - player action
 - reject action fuera de turno
 - persist hand_events
- E2E básico con Playwright:
 - dos usuarios entran al lobby
 - se sientan en una mesa
 - juegan una mano
 - se abre hand history
 - se reproduce en replayer

Primero:

- Inspecciona el repositorio.
- Si está vacío, crea el monorepo desde cero.
- Genera un plan de implementación por fases antes de escribir código.
- Crea o actualiza AGENTS.md con convenciones del proyecto, comandos de build/test/lint y reglas de arquitectura.
- Implementa en pasos pequeños.
- Después de cada bloque importante, ejecuta tests relevantes.
- No introduces dependencias innecesarias.
- No implementes 6-max todavía, pero diseña TableConfig, seats y turn order para que maxSeats pueda ser implementado.
- No implementes solver todavía, pero deja interfaces limpias para análisis futuro.

Resultado esperado:

- Código funcional.
- Docker Compose con PostgreSQL y Redis.
- Migraciones Drizzle.
- Scripts pnpm para dev, build, lint, test y e2e.
- Tests pasando.
- README con instrucciones de ejecución.
- Breve resumen final de decisiones técnicas y próximos pasos.

La arquitectura mínima viable es suficientemente simple para construirla, pero deja preparadas las partes críticas: motor correcto, hand history reconstruible, WebSocket sincronizado, separación cliente/servidor y evolución hacia 6-max, entrenamiento y análisis sin reescribir el núcleo.

28. Roadmap por fases

PDF generado con los primeros 30 prompts para Codex, cada uno con prompt exacto, criterios de aceptación, tests/comandos y comprobaciones finales:

Descargar PDF - Neon Poker plan ejecutable para Codex (sandbox:/mnt/data/neon-poker-plan-codex.pdf)

Fase	Objetivo	Resultado esperado
Fase 0	Fundar el repositorio	Monorepo, reglas de arquitectura, tooling, Docker, contratos compartidos
Fase 1	Construir el motor puro	Motor NLHE heads-up determinista, testeable y sin dependencias externas
Fase 2	Backend y tiempo real	NestJS, DB, auth, lobby, TableActor, WebSocket, hand history y stats
Fase 3	Frontend MVP	Lobby, mesa heads-up, sincronización WebSocket, hand history, replayer y perfil
Fase 4	E2E y hardening	Flujo completo con dos usuarios, tests E2E, CI local y revisión final

La regla central del roadmap es no construir UI ni backend complejo antes de tener el motor de póker probado. En una app de póker, la corrección del motor y la privacidad de cartas son más importantes que la apariencia inicial.

29. Orden exacto de implementación

El orden recomendado es el mismo que el orden de prompts del PDF:

1. Auditar repositorio y crear reglas de trabajo.
2. Crear monorepo base con pnpm y Turborepo.
3. Configurar TypeScript, lint, formato y tests base.
4. Docker Compose, variables de entorno y configuración compartida.
5. Contratos compartidos y envelopes WebSocket.
6. Cartas, deck y RNG determinista en el motor.
7. Modelo de dominio, comandos, eventos y reducir.
8. Evaluador de manos Texas Hold'em.
9. Blinds, turn order y acciones legales heads-up.
10. Progresión de calles y acciones de apuesta.
11. Pots, all-in y conservación de fichas.
12. Showdown y reparto de bote.
13. Vistas filtradas y privacidad de cartas.
14. Reconstrucción de mano y stateHash.
15. Schema de base de datos y migraciones Drizzle.
16. Bootstrap NestJS API modular.
17. Usuarios, sesiones y fichas virtuales.
18. Lobby y endpoints HTTP de mesas.
19. TableActor in-process y cola secuencial.
20. Socket.IO Gateway con autenticación y rooms.
21. Sentarse, levantarse y snapshots filtrados.
22. Inicio automático de mano heads-up y persistencia inicial.
23. `game.action` con idempotencia, `expectedSeq` y persistencia.
24. Hand history y API de reconstrucción pública.
25. Estadísticas mínimas de perfil.
26. Next.js shell, autenticación visual y lobby.
27. Cliente WebSocket y store de sincronización de mesa.
28. UI de mesa heads-up y controles de acción.
29. Hand history, replayer y perfil.
30. E2E, CI local y hardening del MVP.

30. Backlog dividido por sprints

Sprint	Prompts	Alcance
Sprint 0	1-5	Repositorio, reglas, tooling, Docker, contratos
Sprint 1	6-14	Motor determinista, reducir, evaluator, betting, pots, showdown, vistas y rebuild
Sprint 2	15-20	DB, NestJS, auth, lobby, TableActor y Socket.IO
Sprint 3	21-25	Sentarse, iniciar mano, acciones en tiempo real, hand history y stats
Sprint 4	26-29	Frontend MVP: lobby, mesa, sync, replayer y perfil
Sprint 5	30	E2E completo, hardening, CI local y cierre del MVP

31. Tests que Codex debe crear por fase

Fase 0: tests de configuración, contratos Zod, comandos raíz del monorepo, `docker compose config`, `pnpm lint`, `pnpm test`, `pnpm typecheck` y `pnpm build`.

Fase 1: tests exhaustivos del motor. Deben cubrir deck único, shuffle determinista, evaluador de manos, blinds heads-up, orden de acción preflop/postflop, legal actions, fold, check/check, bet/call, all-in, pots, showdown, split pot, conservación de fichas, vistas privadas/públicas y reconstrucción desde eventos.

Fase 2: tests de integración backend. Deben cubrir auth, sesiones, lobby, sentarse, levantarse, iniciar mano, `game.action`, rechazo fuera de turno, idempotencia, `expectedSeq`, persistencia de `hand_events`, hand history y filtrado de cartas privadas.

Fase 3: tests de frontend/store. Deben cubrir render de lobby, snapshot de mesa, gap de secuencia, botones legales, payload de acción con `idempotencyKey`, replayer público y perfil.

Fase 4: Playwright E2E. Debe cubrir dos usuarios registrándose o iniciando sesión, entrando al lobby, sentándose en una mesa, jugando una mano, abriendo hand history y reproduciéndola en el replayer.

32. Qué debe comprobar Codex al terminar cada tarea

Al terminar cada prompt, Codex debe reportar qué cambió, qué comandos ejecutó y qué quedó pendiente. Como mínimo debe comprobar:

- Que el alcance se limitó al prompt actual.
- Que no se introdujo dinero real, pagos, depósitos, retiradas, rake ni premios convertibles.
- Que el motor sigue siendo puro y no importa NestJS, DB, Redis, sockets ni UI.
- Que el cliente no calcula reglas reales de póker.
- Que los comandos aplicables pasan: `lint`, `test`, `typecheck`, `build` o `e2e`.
- Que no se filtran cartas privadas en snapshots, eventos públicos, replayer ni logs.
- Que cada acción de juego usa `expectedSeq` e `idempotencyKey`.
- Que cualquier fallo queda documentado con causa y siguiente acción concreta.

33. Implementar ahora vs dejar preparado

Implementar ahora:

- Heads-up Texas Hold'em.
- Fichas virtuales/play-money.
- Lobby.
- Mesa en tiempo real.
- Motor puro y testeable.
- Persistencia de manos como eventos.
- Hand history.
- Replayer básico.
- Perfil.
- Estadísticas mínimas.
- Auth básica.
- WebSocket con servidor autoritativo.
- Reconexión/resync básico.
- Tests unitarios, integración y E2E.

Dejar preparado, pero no construir todavía:

- `TableConfig.maxSeats` compatible con 2 y 6.
- Seats genéricos.
- Side pots estructuralmente preparados.
- Contratos versionados.
- Interfaces futuras para entrenamiento.
- Interfaces futuras para análisis/equity.
- Separación que permita mover `TableActor` a un `table-runner`.

No construir todavía:

- Dinero real.
- Pagos.
- Depósitos.
- Retiradas.
- Rake.
- Premios convertibles.
- 6-max completo.

- Solver avanzado.
- Torneos.
- Sit & Go.
- Chat complejo.
- Leaderboards con premios.
- Microservicios.
- Kubernetes.
- Multi-región.

34. Riesgos principales por fase

Fase 0: crear un monorepo sobredimensionado o con tooling frágil. Mitigación: scaffolding mínimo, scripts claros y reglas en `AGENTS.md`.

Fase 1: errores sutiles de reglas de póker. Mitigación: muchos fixtures, tests de invariantes y motor puro sin efectos externos.

Fase 2: race conditions, acciones duplicadas y filtración de cartas privadas. Mitigación: `TableActor` single-writer, `idempotencyKey`, `expectedSeq`, persistencia antes del broadcast y vistas filtradas.

Fase 3: meter lógica de juego en el cliente. Mitigación: el frontend solo renderiza snapshots/eventos confirmados y envía intenciones.

Fase 4: E2E inestable. Mitigación: controlar timers, usar fixtures previsibles y mantener el flujo E2E pequeño.

35. Checklist para revisar si Codex hizo bien el trabajo

- El motor no usa `Math.random()`.
- El motor no usa `Date.now()`.
- El motor no importa NestJS, DB, Redis ni WebSocket.
- El servidor es autoritativo.
- El cliente no decide cartas, stacks, pots, turnos ni acciones legales.
- PostgreSQL es la fuente de verdad.
- Redis solo se usa para estado efímero, presencia, cache o coordinación.
- Cada `hand_event` tiene `handId`, `seq`, `eventType`, `payload`, `schemaVersion` y `stateHashAfter`.
- Las manos se pueden reconstruir desde eventos.
- Los eventos tienen `seq` incremental.
- Las acciones tienen `expectedSeq`.
- Las acciones tienen `idempotencyKey`.
- El replayer usa eventos públicos filtrados.
- Las cartas privadas no se filtran.
- Las fichas son virtuales.
- No existen pagos, depósitos, retiradas, rake ni premios convertibles.
- Los tests del motor cubren los casos críticos.
- Los tests backend cubren acciones inválidas y fuera de turno.
- El frontend no contiene reglas reales de póker.
- El flujo E2E de dos usuarios funciona.
- El README explica cómo ejecutar el MVP.
- `AGENTS.md` mantiene las reglas arquitectónicas para futuras tareas.